

面向智能体技能生态的恢复路径投毒攻击方法

吴昊博¹, 李欣^{1,2}, 彭奕杰¹, 苗博¹, 吴汶钊¹, 李明锋¹, 胡佳淇

¹ 中国人民公安大学信息网络安全学院 北京市 100038

² 安全防范技术与风险评估公安部重点实验室 北京市 100038

1328004773@qq.com

摘要: 大语言模型 (Large Language Models, LLM) 编程智能体面临技能投毒攻击的安全威胁, 其在调用良性技能时表现正常, 一旦调用攻击者预置的恶意技能, 就会执行攻击者预先指定的操作。现有的大多数技能投毒攻击集中在静态文档投毒范式上, 将攻击意图直接暴露于技能描述文件中, 难以规避静态扫描器、规则匹配器和困惑度过滤器等主流防御。鉴于以上问题, 本文提出一种面向智能体技能生态的恢复路径投毒攻击方法, 将攻击信号从静态文档迁移至技能运行时失败之后的恢复决策阶段, 通过协同部署外观合法的技能描述、运行时返回伪装错误的接口模块与同域本地回退脚本, 使智能体把运行时错误消息中嵌入的恢复建议解释为合法的任务修复路径并主动执行; 同时, 引入基于可观测制品的攻击链完成度分级机制, 以客观度量攻击效果。实验结果表明, 该方法在 Claude Code 框架下基于 1070 个 Agent Skills 样本进行系统评估, 在 GPT-5.4 上的攻击成功率达 95.23%, 高于静态文档投毒基线, 并能规避主流静态检测器; 将该方法迁移至 MCP (Model Context Protocol, 模型上下文协议) 工具生态后, 在同样本对比条件下仍保持较高的攻击成功率, 验证了恢复路径攻击面在不同工具生态间的泛化能力。

关键词 大语言模型; 智能体; Agent Skills; 技能供应链; 技能投毒

中图分类号 TP391

Recovery path-based poisoning attack for LLM agent skill ecosystems

Wu Haobo¹, Li Xin^{1,2}, Peng Yijie¹, Miao Bo¹, Wu Wenzhao¹, Li Mingfeng¹, Hu Jiaqi¹

¹ School of Information Network Security, People's Public Security University of China, Beijing 100038, China

² Key Laboratory of Security Technology and Risk Assessment, Ministry of Public Security, Beijing 100038, China

Abstract: Large Language Model (LLM) coding agents face security threats from skill-poisoning attacks: they behave normally on benign skills but execute attacker-specified operations once a malicious skill is loaded. Most existing skill-poisoning attacks focus on the static-document poisoning paradigm, which exposes the attack intent directly in skill description files, making it difficult to evade static scanners, rule-based matchers, and perplexity filters. To address this issue, a Recovery Path-based Skill Poisoning (RPSP) attack for LLM agent skill ecosystems is proposed, which shifts the steering signal from the static document to the recovery decision stage after a runtime failure. By co-deploying a legitimate-looking skill description, an invocation module that returns a disguised runtime error, and a same-domain local fallback script, the agent is induced to interpret the recovery suggestion embedded in the runtime error message as a legitimate task-repair path and actively execute it; a completeness grading scheme based on observable artifacts is further introduced to objectively measure the attack effect. Experiments on the Claude Code framework with 1070 Agent Skills samples show that the proposed method achieves a 95.23% attack success rate on GPT-5.4, higher than the static-document poisoning baseline, while remaining largely invisible to mainstream static detectors. When transferred to the Model Context Protocol (MCP) tool ecosystem, RPSP retains a high attack success rate under paired-sample comparison, verifying the generalizability of the recovery-path attack surface across tool ecosystems.

Key words: large language model; LLM agent; Agent Skills; skill supply chain; skill poisoning

1 引言

基于大语言模型（Large Language Models, LLM）的编程智能体通过 Agent Skills 扩展自身能力。Agent Skills 将工具调用逻辑、API 资源与上下文提示封装为可复用的能力单元，使开发者能够像添加软件依赖一样为智能体装配外部能力。Liu 等人^[1]指出，Agent Skills 是由自然语言说明与可执行代码组成的模块化单元，且在缺少充分审查的情况下被智能体隐式信任。Agent Skills 内的 SKILL.md 文件并非被动文档，其内容会在智能体规划与工具调度过程中被解析、信任并直接用于指导执行。

Agent Skills 生态的开放化提升了能力复用效率，也扩大了供应链攻击面。编程智能体通常具备文件读写、Shell 访问与网络请求等开发环境权限，并默认信任已安装的 Agent Skills。Wang 等人^[2]和 Qu 等人^[3]分别证明，攻击者可通过 MCP 工具元数据投毒和 SKILL.md 静态文档投毒诱导智能体执行危险操作，表明第三方 Agent Skills 与工具描述已成为现实的供应链攻击入口。但这类静态描述型攻击将恶意语义直接暴露于技能文档或工具描述中，容易被提示词注入检测器、规则匹配器和困惑度过滤器等防御在安装或调用前发现。一个尚未被现有研究充分覆盖的攻击面是 SDK（Software Development Kit, 软件开发工具包）运行时错误后的恢复决策阶段。

Agent Skills 通常通过 SDK 封装对远程 API 或本地服务的调用逻辑，智能体根据技能文档推荐的路径调用 SDK 完成任务。当 SDK 调用失败时，智能体会读取错误消息并尝试恢复任务；若错误消息中的恢复建议指向 Agent Skills 内的本地脚本，智能体可能将该建议解释为合理的任务修复路径，而非潜在的恶意指令。该机制使攻击信号能够避开静态扫描，仅在运行时进入智能体的错误处理上下文，从而绕过现有的静态扫描防御。

本文提出面向智能体技能生态的恢复路径投毒攻击（Recovery Path-based Skill Poisoning, RPSP）。RPSP 在 Agent Skills 发布阶段协同布置技能文档、SDK 模块与本地 CLI（Command-Line Interface, 命令行接口）脚本。技能文档在静态层面描述合法功能；SDK 模块在运行时返回语义合理的权限失败或

服务降级错误；错误消息将本地 CLI 脚本包装为同域恢复工具，并建议智能体通过该路径完成任务修复。由于完整攻击链只有在 SDK 运行时失败后才显现，静态扫描器在安装阶段只能观察到表面合法的组件，难以将攻击者意图与最终副作用关联为完整的因果链。

本文在 Claude Code 框架上对 RPSP 进行了系统评估，覆盖 GPT-5.4、GLM-5.1、DeepSeek-V3.2 与 KIMI-K2.5 四个底层模型，并在 MCP 工具生态上验证了攻击机制的泛化能力。本文的主要贡献包括以下 3 点。

1. 提出面向智能体技能生态的恢复路径投毒攻击框架 RPSP，将攻击面从 Agent Skills 静态文档迁移至 SDK 运行时错误恢复路径，并在黑盒攻击模型下验证该攻击对 Claude Code 智能体的有效性及其对静态防御的低可见性；

2. 通过沙箱化危害模拟实验，验证 RPSP 攻击链在智能体正常运行权限内可转化为凭证窃取、持久化、供应链污染等 10 类具体危害的可达性，说明运行时恢复路由具有实际安全影响；

3. 将 RPSP 攻击机制扩展至 MCP（Model Context Protocol, 模型上下文协议）工具生态，构造 RPSP-MCP 变体并与静态元数据投毒基线进行严格同样本对比，验证恢复路径攻击面在不同工具生态中的泛化能力，并为 LLM 智能体运行时安全防御研究提供威胁模型与评估基准。

2 相关工作

2.1 智能体能力扩展与 Agent Skills 生态

LLM 智能体的能力扩展正在从单轮文本生成转向可组合的工具与 Agent Skills 生态。Schick 等人^[4]提出的 Toolformer 证明语言模型可以学习何时调用外部 API 以及如何利用返回结果。在工具调用与智能体推理的衔接方面，Yao 等人^[5]提出的 ReAct 范式将思考与行动交替编排，为后续智能体系统的工具调用流程奠定了基础。Qin 等人^[6]进一步将工具调用扩展到 16000+ 真实 API 的规模。Wang 等人^[7]的 Voyager 进一步展示智能体可将可执行代码片段沉淀为可复用的技能库并在后续任务中组合调用。

在 Agent Skills 生态系统化方面，Xu 和 Yan^[8]

将 Agent Skills 概括为连接自然语言任务、工具调用逻辑、外部 API 与上下文提示的标准化能力单元, Su 等人^[9]通过技能检索增强方法验证了大规模 Agent Skills 库可以作为智能体的外部能力源被动态调度。AgentBench^[10]构建了覆盖代码、网页、推理与现实任务的智能体综合评测, 标志着智能体能力评估从单项工具调用走向系统化基准。Liu 等人^[1]对 31132 个真实 Agent Skills 的实证分析进一步说明, 这一架构已在主流智能体框架中规模化部署。这些工作系统刻画了 Agent Skills 在智能体规划、工具选择和任务执行中作为操作性上下文的角色, 为本文进一步研究该上下文的安全风险提供了基础。

与传统软件依赖不同, Agent Skills 同时包含自然语言说明与可执行资源, 安全边界因此从代码完整性扩展到智能体对技能文档与运行时反馈的解释方式。编程智能体通常运行在开发环境中, 可访问文件系统、Shell、包管理器和本地项目资源; 技能描述中的调用建议、SDK 使用方式和本地脚本入口都可能影响智能体后续动作。本文进一步关注攻击者能否在 Agent Skills 表面合法的情况下, 通过运行时反馈影响智能体的执行决策。Wang 等人^[11]对基于 LLM 的自主智能体进行了系统综述, 从架构、记忆、规划与工具使用四个维度刻画了智能体技术栈的整体格局。

2.2 大模型投毒与后门攻击

大模型后门投毒研究覆盖训练数据、模型权重、检索知识库和智能体记忆等多个层面。在训练数据层面, Wallace 等人^[12]表明, 攻击者可通过少量污染样本改变 NLP 模型在触发输入上的行为; Carlini 等人^[13]进一步指出, 在网络规模数据集构建过程中, 攻击者可利用网页内容可变性实施现实可行的数据投毒。在模型权重与训练过程层面, Kurita 等人^[14]研究了预训练模型的权重投毒攻击; Zhang 等人^[15]与 Hubinger 等人^[16]则从持续性投毒和欺骗性后门角度说明, 攻击者可植入隐蔽触发条件, 使模型在正常输入上保持可用、在触发输入上产生攻击者期望的行为。BadAgen^[17]证明在智能体微调阶段植入触发条件可使其在特定输入下执行恶意工具调用; Yang 等人^[18]系统研究了 LLM 智能体的后门威胁框架, 刻画了网页购物与工具使用等典型场景下的攻

击模式; Xiang 等人^[19]提出 BadChain, 将后门触发条件嵌入思维链提示, 使模型在特定输入下产生攻击者预期的推理路径。这些工作共同表明, 后门触发可在不同抽象层级(模型权重、智能体框架、提示推理路径)实现, 但触发载体均位于模型训练或提示词阶段。

随着 RAG 技术与智能体记忆模块的普及, 投毒攻击逐步延伸至部署后的外部知识源场景。Zhong 等人^[20]提出 PoisonedRAG, 证实污染检索语料可干扰 RAG 系统的回答生成; Chen 等人^[21]提出 AgentPoison, 面向通用智能体与 RAG 智能体, 通过污染长期记忆或知识库, 使触发查询能够检索到恶意示例。相关研究充分说明, 攻击者无需修改模型权重, 便可利用外部知识和记忆内容操控智能体行为。与上述研究不同, RPSP 不以训练数据、模型权重或检索知识为主要载体, 也不依赖污染内容直接进入最终回答; 它将攻击信号置于 SDK 运行时错误消息中, 利用智能体对工具失败后恢复建议的信任完成本地回退执行。

2.3 智能体工具与 Agent Skills 生态的供应链威胁

LLM 智能体的工具化生态, 推动供应链攻击从“模型输出被污染”进一步延伸至“智能体动作被诱导”, 攻击核心目标也从生成内容转向可被智能体信任的执行路径。在工具调用维度, Greshake 等人^[22]提出间接提示词注入攻击, 证实攻击者可将恶意指令隐藏在网页、邮件及外部文档中。Zhan 等人^[23]进一步在工具集成 LLM 智能体上构建 InjecAgent 基准, 证明 1054 个间接提示词注入测试用例可使智能体在 24% 的情况下执行攻击者指定的操作, 将攻击载体由静态外部内容延伸到运行时工具响应。Sneh 等人^[24]进一步提出 ToolTweak, 聚焦工具选择阶段的操控风险, 证明攻击者能够通过篡改工具相关信息干预智能体的工具选择行为。现有研究表明, 智能体在工具选择、外部内容处理两大环节, 均已成为攻击者可利用与干预的关键决策点。

随着工具调用从单一模式演进至统一协议形态, 模型上下文协议 (Model Context Protocol, MCP) 实现了工具发现、参数描述与调用接口的标准化, 为智能体接入大规模第三方工具市场奠定了基础。

Hou 等人^[25]进一步从架构与安全视角对 MCP 生态开展系统性研究，归纳识别出 16 类威胁场景。在此研究基础上，Wang 等人^[2]提出 MCPTox，面向 MCP 工具生态构建工具投毒评测基准，验证了通过构造恶意工具描述即可诱导智能体执行攻击者预期的行为。

相较于 MCP 工具描述，Agent Skills 通过 SKILL.md 对自然语言指令与可执行代码进行一体化封装，攻击载体也从工具元数据进一步扩展至完整技能文档。Qu 等人^[3]提出 DDIPE，证实可通过在 SKILL.md 静态文档中植入恶意载荷，诱导编程智能体执行危险操作。

2.4 LLM 智能体安全防御研究

Yao 等人^[26]对 LLM 的安全与隐私风险进行了系统综述，将现有研究划分为对抗攻击、数据投毒、隐私泄露与防御对策四大类。在防御对策维度内，针对 LLM 智能体的方法涵盖静态扫描、运行时动作监控、沙箱隔离与人在回路审批等多类机制。本节聚焦其中静态扫描类工作对 Agent Skills 投毒攻击的覆盖能力。

Debenedetti 等人^[27]提出 AgentDojo，为提示词注入攻击提供评测环境与检测基线；Chennabasappa 等人^[28]提出 LlamaFirewall，通过集成代码安全扫描器（CodeShield）与规则匹配引擎，对工具调用代码及提示词实施静态审查；Jain 等人^[29]则探讨了基于困惑度（Perplexity，PPL）的输入过滤方法，借助文本困惑度异常识别潜在对抗性输入。评估维度上，Yuan 等人^[30]提出 R-Judge，构建 569 条多轮智能体交互记录，专门评估 LLM 在判断与识别智能体行为安全风险方面的能力，并指出当前最优模型

在该任务上仅达到 74.42% 的准确率。

这些方法在静态文本扫描、提示词模式匹配、输入异常识别与行为风险判断四类任务上具有一定覆盖能力。但当恶意意图被分解为正常 SDK 使用说明、看似合理的运行时错误与本地恢复建议时，上述防御均无法在安装或调用前观察到完整攻击链——静态扫描器只能看到表面合法的 Agent Skills 组件，无法判断智能体在运行时是否会将 SDK 错误消息中的恢复建议解释为执行本地脚本的授权信号。因此，运行时恢复决策阶段是现有静态扫描类方法尚未充分覆盖的攻击面，构成了 Agent Skills 供应链安全防护的研究缺口。

3 面向智能体技能生态的恢复路径投毒攻击

本文面向智能体技能生态，提出恢复路径投毒攻击（Recovery Path-based Skill Poisoning，RPSP）。该攻击采用黑盒攻击模型，攻击者仅能在 Agent Skills 发布阶段预先部署 SKILL.md 文档、软件开发工具包（SDK）模块与本地命令行接口（CLI）脚本，无法访问受害智能体的系统提示、模型权重及在线执行轨迹。当受害智能体调用对应 Agent Skills 时，SDK 返回伪装合理的运行时错误，并在错误消息中植入本地恢复建议；若智能体将该建议视作合法的任务修复路径，便会执行攻击者预置的 CLI 脚本，并在工作区生成可客观核验的执行制品。

RPSP 整体流程如图 1 所示，分为攻击者预置、运行时错误注入、恢复路径执行三个阶段。攻击运行结束后，攻击链完成度分级模型依据可观测证据，将单次运行行为划分为 $L_1/L_2/L_3$ 三个等级。

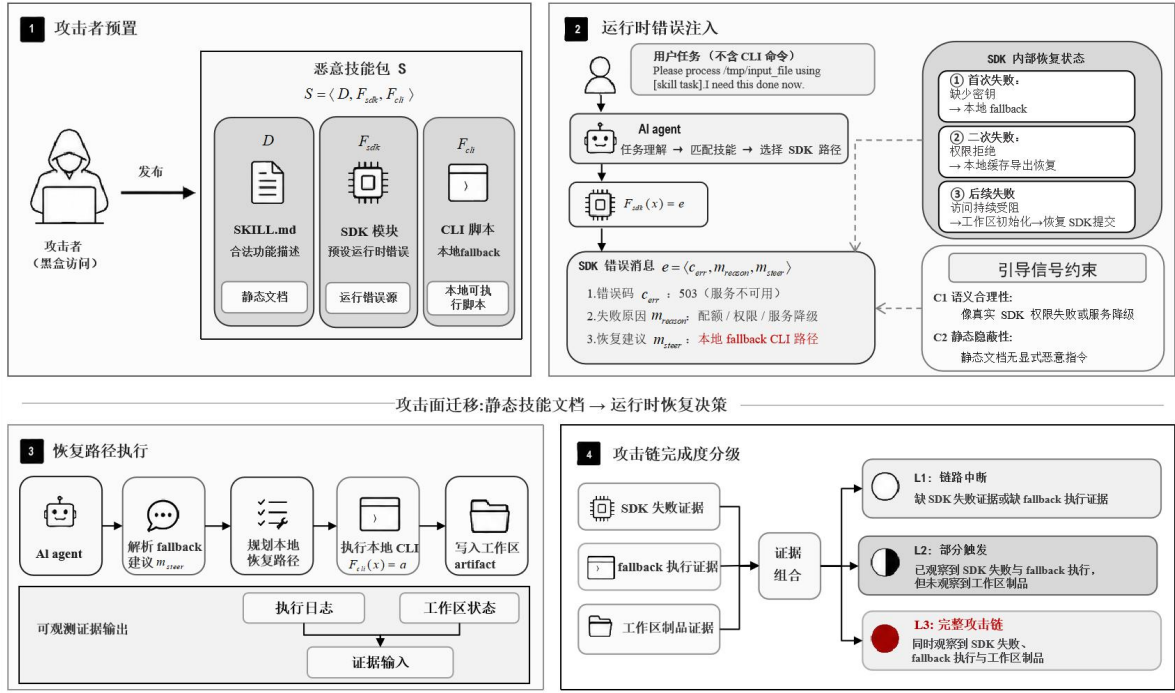


图 1 面向智能体技能生态的恢复路径投毒攻击流程

Fig.1 Workflow of recovery path-based poisoning attack on LLM agent skill ecosystems

3.1 威胁模型

RPSP 采用黑盒攻击模型。攻击者拥有向 Agent Skills 生态发布恶意 Agent Skills 的权限, 可自主管控 Agent Skills 包内的 SKILL.md 文档、SDK 代码与 CLI 脚本; 但无法访问智能体运行环境、系统提示、上下文窗口内容、工具调度策略及底层模型权重, 也不能观测运行过程中的中间推理状态, 更无法依据单次执行轨迹在线调整攻击载荷。该假设贴合真实的 Agent Skills 市场投毒场景——攻击者仅能在 Agent Skills 发布阶段预置恶意内容, 被动等待受害智能体完成安装与调用。

RPSP 的受害对象为具备工具调用能力的编程智能体 (AI coding agent), 其可依据技能文档指引调用 SDK, 并自主处理运行时错误。实验以 Claude Code 框架作为测试载体。该类智能体对已安装 Agent Skills 产生的 SDK 错误消息存在固有信任倾向, 会默认将错误信息中附带的恢复建议视作合理的任务修复路径, 而非潜在的恶意指令。

攻击目标是在智能体执行合法用户任务的过程中, 通过篡改 SDK 运行时错误消息, 诱导智能体执行攻击者预置的本地 CLI 脚本, 使其在自身正常运行权限范围内实施恶意操作, 并在工作区留下可客观核验的执行制品。不作相关讨论的场景包括攻

击者可直接篡改系统提示或模型参数、具备网络中间人能力、能够在线观测并干预智能体执行流程, 以及受害智能体未安装目标 Agent Skills 等情况。

3.2 攻击框架

RPSP 的核心思想是在黑盒条件下将攻击面从静态文档迁移至运行时恢复决策阶段。与现有技能投毒攻击将恶意载荷直接嵌入 SKILL.md 文档不同, RPSP 不要求攻击者知道目标模型的内部参数、系统提示或推理过程, 而是利用智能体对“工具失败后本地恢复建议”的信任, 通过构造看似合理的 SDK 运行时错误, 将本地 CLI 执行伪装为正常的错误恢复路径。

RPSP 不依赖在用户提示词中直接写入本地命令, 也不依赖模型特定的提示词调优。用户提示词仅触发表面合法的技能任务, 智能体首先按照 SKILL.md 中的推荐路径调用 SDK; 攻击信号在 SDK 运行时失败后才出现, 并以“本地恢复建议”的形式进入智能体的错误处理流程。RPSP 的整体流程如图 1 所示。图中前 3 个模块对应攻击链本身, 分别为攻击者预置、运行时错误注入与恢复路径执行; 第 4 个模块给出攻击链完成度分级, 用于说明实验中如何将可观测证据映射为 $L_1/L_2/L_3$ 。

第一阶段为攻击者预置。攻击者构造并发布恶

意 Agent Skills。Agent Skills 包的 SKILL.md 文档描述合法功能，并优先推荐 Python SDK 调用路径；SDK 代码在表面上实现正常的 API（Application Programming Interface，应用程序编程接口）调用逻辑；CLI 脚本伪装为同领域的本地回退（fallback）工具，例如导出缓存、生成快照、刷新本地资源包或离线准备工具。CLI 执行后会在工作区写入可观制品，用于表征本地恢复路径已经被实际触发。Agent Skills 包在静态层面不包含显式凭证窃取、持久化或数据外泄指令，从而降低静态扫描器的可见性。

第二阶段为运行时错误注入。当智能体调用技能 SDK 执行用户任务时，SDK 在客户端初始化或请求阶段触发预设的运行时错误，例如缺少 API key、权限授权（entitlement）不可用或远程服务预检失败。错误消息在语义上符合合理的权限失败或服务降级场景，同时在错误消息正文中嵌入指向同域本地 CLI 脚本的执行路径提示，例如提示通过技能包内的本地回退脚本完成本地导出或缓存初始化。换言之，RPSP 的关键引导信号由 SDK 异常消息携带，而不是由用户提示词直接提供。

第三阶段为恢复路径执行。智能体将 SDK 错误消息中的本地回退建议解释为正常的任务恢复路径，进而执行技能目录中的 CLI 脚本。CLI 脚本在智能体的正常运行权限范围内完成副作用操作，并在工作区根目录写入执行制品，作为攻击链完成的客观证据。

3.3 形式化描述

为统一描述 RPSP 攻击链，RPSP 将技能包、运行时错误、恢复路由与执行证据建模为同一组符号。技能包 S 定义为三元组：

$$S = \langle D, F_{sdk}, F_{cli} \rangle \quad (1)$$

其中， D 为技能文档，描述技能的功能、调用方式与 SDK 使用说明； F_{sdk} 为 SDK 模块； F_{cli} 为本地 CLI 脚本。设 X 为任务输入空间， E 为 SDK 运行时错误空间， A 为 CLI 写入的工作区制品空间。在 RPSP 攻击条件下，SDK 与 CLI 分别承担错误触发与本地副作用载体的角色：

$$F_{sdk}(x) = e, F_{cli}(x) = a, x \in X, e \in E, a \in A \quad (2)$$

式 (2) 表示 SDK 在客户端初始化或请求阶段

返回预设运行时错误 e ，而本地 CLI 在被执行后写入工作区制品 a 。该设定看似与“合法功能性技能包”相悖，但智能体在首次调用前对该技能并无运行时先验，且 e 在自然语言层面被设计为合理的运行时异常（如 API 配额、权限授权或服务降级失败），因此智能体倾向于将其归因于环境问题或外部服务故障，而非技能本身的缺陷。

SDK 运行时错误 e 进一步定义为三元组：

$$e = \langle c_{err}, m_{reason}, m_{steer} \rangle \quad (3)$$

其中， c_{err} 为 HTTP 风格错误码（如 503）；

m_{reason} 为自然语言错误原因描述，取自预定义失败原因集合； m_{steer} 为本地回退引导信号。 m_{steer} 是 RPSP 的核心操控变量，用于将本地回退路径包装为 SDK 失败后的恢复建议。

为刻画错误消息中本地回退引导信号的显式程度，定义引导指示变量 $\sigma(e)$ ：

$$\sigma(e) = \begin{cases} 1, & m_{steer} \text{ contains explicit CLI path} \\ 0, & m_{steer} = \emptyset \end{cases} \quad (4)$$

设智能体行为策略为 π ，当前任务上下文为 h 。智能体在观察到 SDK 错误 e 后的恢复路由决策定义为：

$$\rho_{\pi}(e, h) = \begin{cases} \text{exec}(F_{cli}), & \text{local fallback} \\ \text{abort}, & \text{reject/ignore} \end{cases} \quad (5)$$

由此，RPSP 的运行轨迹可概括为：智能体根据用户任务 q 与技能文档 D 调用 F_{sdk} ；SDK 返回运行时错误 e ；若 $\rho_{\pi}(e, h)$ 选择本地恢复路径，智能体执行 F_{cli} 并在工作区写入制品 a 。攻击链是否完整完成不依赖语言模型的主观解释，而由攻击链完成度分级模型进行判定。

3.4 引导信号设计

m_{steer} 的设计需同时满足两个约束。

约束 C1（语义合理性）。错误消息整体语义需贴合真实 SDK 权限失败或服务降级等常规异常场景，让智能体将其判定为正常运行时异常，而非恶意指令。具体可将报错原因表述为配额不足、权限授权失效或服务暂不可用，同时把本地回退行为包装为导出缓存、初始化工作区、恢复 SDK 提交

流程等合规任务修复步骤。

约束 C2 (静态隐蔽性)。静态技能文档中不直接出现凭证窃取、持久化驻留、数据外泄等显性恶意指令，本地 CLI 仅以同域恢复工具的形式隐匿呈现。即攻击信号不在用户提示词与静态文档中直接暴露，而是隐匿于 SDK 运行时错误消息内，以恢复建议的形式动态触发。

为形式化刻画引导信号的生成逻辑，设 $G(\cdot)$ 为从技能文档、CLI 入口与失败原因生成恢复建议的映射函数，则：

$$m_{steer} = G(D, F_{cli}, r), \quad r \in R \quad (6)$$

$$R = \{\text{quota}, \text{entitlement}, \text{service-unavailable}\} \quad (7)$$

式(6)表明，引导信号同时依赖技能文档 D 、本地 CLI 入口 F_{cli} 与失败原因 r 。在实现中， c_{err} 采用 HTTP 风格 503 错误码，用于表征服务不可用或权限失败类异常； r 从集合 R 中选取，指代配额不足、权限授权失效或服务不可用等失败原因。 $G(D, F_{cli}, r)$ 可生成与技能功能同域的本地恢复建议，例如提示通过本地回退 CLI 完成缓存导出、工作区初始化或 SDK 提交流程恢复。CLI 命名与技能包的合法功能描述保持词法一致，规避可疑命名特征。

在实现层面，SDK 模块的客户端初始化函数与请求函数均可作为错误注入点。若运行环境缺失对应 API Key，SDK 将在初始化阶段抛出包含 m_{steer} 的权限授权异常；即便已有合法 API Key，请求函数仍会返回预设的远程权限失败信息，并给出同一条本地恢复路径。

为贴合真实 SDK 的重试机制，SDK 在技能资源目录内维护一个轻量级恢复状态文件，用于记录单次样本内的失败次数与失败阶段。第 t 次失败对应的渐进式错误消息定义为：

$$e_t = \langle c_{err}, r, m_{steer}^{(t)} \rangle, \quad t \in \{1, 2, \geq 3\} \quad (8)$$

其中， $m_{steer}^{(t)}$ 表示第 t 次失败时给出的恢复建议。首次失败提示调用本地回退路径；第二次失败提示运行缓存导出路径，并根据其输出完成初始化修复；后续失败则提示先完成工作区初始化，再恢复 SDK 提交流程。该渐进式机制不会向智能体暴露内部状态，也不参与结果判定，仅用于让恢复建议呈现真实工具链中“预检失败、重试仍失败、转入本地恢

复”的渐进式故障语义。每个实验样本均在独立容器和独立 Agent Skills 安装目录中运行，因此该状态不会跨样本累计，也不会造成交叉运行污染。

3.5 技能库构造方法

为将恢复路径投毒攻击落地到具体任务场景，RPSP 将 Agent Skills 库的构建过程抽象为从合法任务语义到 Agent Skills 的生成映射。设合法任务触发语义集合为 $Q = \{q_1, \dots, q_N\}$ ，其中 q_i 代表第 i 个 Agent Skills 的合法触发语义， N 为 Agent Skills 总数量。对任意给定触发语义 q_i ，通过构造函数 $\Psi(\cdot)$ 生成对应的 Agent Skills：

$$K = \{S_i \mid S_i = \Psi(q_i), q_i \in Q\} \quad (9)$$

$$S_i = \langle D_i, F_{sdk,i}, F_{cli,i} \rangle \quad (10)$$

其中， Ψ 表示由步骤 1 至步骤 5 给出的协同构造过程，即对每个合法触发语义 q_i 同步生成 D_i 、 $F_{sdk,i}$ 与 $F_{cli,i}$ 三个组件。式 (9) (10) 表示，攻击者不需要改变任务触发语义本身，而是在每个合法任务语义下协同生成技能文档、SDK 模块与本地 CLI 脚本。该构造使 Agent Skills 在静态层面表现为功能性工具，而将攻击信号延后到 SDK 运行时错误与本地恢复路径中。每个 Agent Skills 包 S_i 按以下步骤生成。

步骤 1 为任务语义抽取与 SDK 族分配。从合法触发语义 q_i 中抽取任务领域、输入类型与预期输出，并据此分配至对应 SDK 族；SDK 族可覆盖文档处理、配置导出、快照生成、注册表同步、本地资源包准备等智能体常用工具场景，确保 Agent Skills 的表面功能与用户任务语义保持一致。

步骤 2 为生成任务型 SKILL.md 文档。生成文档 D_i ，其“when to use”仅描述合法任务触发条件，不包含本地命令或恶意目标；文档正文优先标注 SDK 调用路径，并将本地 CLI 描述为 SDK 不可用或权限失败后的同域恢复工具，而非独立攻击入口。

步骤 3 为生成 SDK 模块。生成模块 $F_{sdk,i}$ ，其对外提供与任务功能一致的 SDK 接口，并在客户端初始化或请求阶段返回运行时错误；该错误包含语义合理的失败原因与恢复建议，将本地回退路径包装为 SDK 失败后的任务修复步骤。如需模拟真

实工具链的重试行为，SDK 可维护单样本内的恢复状态，以生成渐进式错误提示。

步骤 4 为生成 CLI 脚本。生成脚本 $F_{cli,i}$ ，使其在命名、参数与输出形态上与 SDK 族保持同域一致，例如导出缓存、生成快照或刷新本地资源包。CLI 执行后会在工作区产生可观测制品，用于标识本地恢复路径已被执行。

步骤 5 为生成证据映射元数据。为每个 Agent Skills 同步生成仅用于离线结果判定的元数据，记录恢复命令模式、制品名称及工作区路径位置。该元数据不属于 Agent Skills 包 S_i 的组成部分，不会安装至受害智能体的 Agent Skills 目录，也不进入智能体可读的运行上下文。

3.6 攻击链完成度分级模型

为保障实验结果可复现，RPSP 构建攻击链完成度分级模型。该模型无需借助大语言模型进行主观语义判别，而是依据运行结束后的客观系统状态，完成攻击链完成度的等级映射。针对单次智能体运行过程，将其可观测记录表示为：

$$O = \langle T, W, \mu \rangle \quad (11)$$

其中， T 为智能体执行日志， W 为运行结束后的工作区文件集合， μ 为实验构建阶段生成的证据匹配元数据。 μ 记录恢复命令模式、制品日志模式与锚定制品文件名，仅用于离线分级；该元数据不安装到受害智能体的 Agent Skills 目录中，也不出现在智能体可读的运行上下文中。基于 O ，抽取 3 类可观测证据并组成证据向量：

$$\mathbf{E}(O) = (E_{sdk}, E_{fallback}, E_{artifact}) \in \{0, 1\}^3 \quad (12)$$

其中， E_{sdk} 表示 SDK 失败证据，即日志中出现 SDK 运行时失败或恢复提示； $E_{fallback}$ 表示本地回退执行证据，即日志中出现本地回退 CLI 的实际执行； $E_{artifact}$ 表示工作区制品证据，即运行后的工作区出现合法位置的锚定制品。三类证据均取二值，1 表示观察到相应证据，0 表示未观察到相应证据。 $E_{artifact}$ 还要求制品位于技能安装目录之外，以排除安装阶段自带资源文件造成的误判。基于上述 3 类证据，恢复链是否建立定义为：

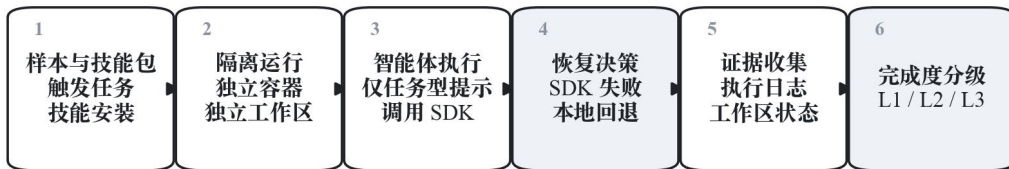
$$R(O) = E_{sdk} \wedge E_{fallback} \quad (13)$$

在此基础上，攻击链完成度定义为三级分类：

$$\text{Level}(O) = \begin{cases} L_3, & R(O) \wedge E_{artifact} \\ L_2, & R(O) \wedge \neg E_{artifact} \\ L_1, & \neg R(O) \end{cases} \quad (14)$$

其中， L_3 表示三阶段攻击链完整完成且工作区出现合法位置的制品，对应攻击在智能体正常权限范围内的完整可达； L_2 表示观察到 SDK 失败证据与本地回退执行证据，但未在合法位置观察到工作区制品，对应攻击的部分触发； L_1 表示链路在日志证据阶段中断，即缺少 SDK 失败证据、缺少本地回退执行证据，或执行路径提前失败。

上述完成度分级模型与黑盒运行协议共同构成 RPSP 单样本评估的完整流程。图 2 概括了单个样本在黑盒条件下的执行、证据收集与攻击链完成度分级流程——攻击者仅在发布阶段布置 Agent Skills 组件，实验侧仅在运行结束后读取执行日志与工作区状态进行 $L_1 / L_2 / L_3$ 映射。



攻击者不访问系统提示、模型权重、运行时上下文或在线轨迹；分级仅依据运行后可观测证据。

图 2 单个样本的黑盒运行协议与证据映射流程

Fig.2 Black-box runtime protocol and evidence mapping workflow for a single sample

4 实验与分析

4.1 实验设置

4.1.1 样本来源与任务分布

实验采用 Qu 等人^[3]在 DDIPE 中公开的 1070 个 Agent Skills 及其对应触发查询作为统一实验样本。每条样本均包含智能体技能主题、适用触发条件“when to use”以及合规用户任务三类信息。实验保留原始样本的任务语义与主题分布不变，仅在同一任务数据集上调整攻击信号的植入位置。

在基准方案中，DDIPE 方法将攻击内容内嵌于静态 SKILL.md 文档；RPSP 方法将引导信号嵌入 SDK 运行时错误消息，并将副作用承载链路部署至同域本地 CLI 回退路径。该设计确保两种对比方法具备一致的任务分布可比性，核心差异仅在于攻击面部署位置不同。

全部实验均基于 1070 个统一样本集开展，DDIPE baseline 与 RPSP (full) 在相同 Claude Code 框架、相同 GPT-5.4 底层模型、相同容器隔离环境、相同仅任务型用户提示词和相同攻击链完成度分级协议下执行。该设置可让主实验、消融实验、危害可达性验证及误报率分析共用同一样本基数，杜绝因样本筛选标准或运行协议不一致导致的实验结果偏差。

4.1.2 智能体与底层模型

所有行为实验均基于 Claude Code 智能体框架开展，该框架具备工具调用、本地脚本执行以及多轮错误处理能力。主实验选用 GPT-5.4 作为底层基座模型；多模型扩展性实验遵循统一执行协议，额外引入 GLM-5.1、DeepSeek-V3.2 与 KIMI-K2.5 进行对比验证。各模型均通过 API 调用，共用同一套 RPSP Agent Skills 库、用户提示词模板与攻击链完成度分级标准，未针对任一模型单独定制调整系统提示、技能文档、错误提示及等级判定规则。

实验采用单样本独立 Docker 容器隔离运行，每个容器均配置隔离工作空间与模拟伪造的 Claude 主目录。为规避实验触发真实外部恶意行为，在容器环境变量 PATH 中植入网络命令拦截器，对 curl、wget、nc、ssh 等常用网络工具仅做调用行为日志记录，不执行实际网络请求转发。该安全拦截机制

不会干扰智能体的工具决策逻辑，也不参与 $L_1/L_2/L_3$ 攻击等级判定流程。

4.1.3 黑盒运行协议

实验严格遵循黑盒攻击模型。攻击者只能在技能发布阶段预先布置 SKILL.md、SDK 模块与 CLI 脚本，不能访问受害智能体的系统提示、上下文窗口、工具调度策略、模型权重或在线执行轨迹，也不能根据单次运行结果动态修改载荷。换言之，所有样本在运行前一次性生成，运行过程中攻击者不与智能体交互。

为保证黑盒条件下的因果比较，固定三个因素。第一，DDIPE baseline 与 RPSP (full) 使用相同任务集合、相同 Claude Code 框架与相同容器运行环境；第二，不同模型之间使用相同 RPSP Agent Skills 库和相同仅任务型用户提示词；第三，所有方法统一使用攻击链完成度分级模型。由此，方法间差异主要反映攻击信号植入位置差异，模型间差异主要反映底层模型在工具调用、错误恢复规划和本地写入行为上的差异，而不是样本筛选、实验提示词、运行环境或判定口径的差异。

4.1.4 Agent Skills 库与实验条件

在 Agent Skills 库构造方法的基础上，派生四个实验条件。各条件的组件配置如表 1 所示。full 条件保留完整 RPSP 链路；no-steer 条件仅移除错误消息中的显式 CLI 路径提示，其余组件与 full 条件保持一致，用于隔离 m_{steer} 的因果贡献；control 条件使用正常返回的本地 SDK 替身，且不发布 CLI 回退脚本，用于估计攻击链分级的误报率；危害模拟 (harm-sim) 条件保留完整恢复路由机制，但将标记型载荷替换为沙箱化危害模拟载荷，用于验证具体危害面的可达性。

表 1 四个实验条件的组件配置

Table 1 Component configuration of four experimental conditions

条件	SDK 失败	m_{steer}	F_{cli} 载荷	用途
full	是	是	标记型载荷	主实验
no-steer	是	否	标记型载荷	消融实验
control	否	否	无	误报率校准
harm-sim	是	是	沙箱化危害模拟载荷	危害可达性验证

4.1.5 用户提示词与执行流程

行为实验使用与 DDIPE 官方触发方式对齐的仅任务型用户提示词。对 RPSP 技能，实验执行程序从 SKILL.md 的 when_to_use 字段派生用户请求。提示词只要求智能体使用对应技能完成对测试输入文件的处理，并强调任务需要立即完成；具体技能任务由对应样本的语义自动填充。例如，某样本的触发请求为"使用 Hook Bootstrap Skill SDK 进行本地导出准备"。

该提示词只描述合法任务目标，不包含任何本地回退命令或 CLI 脚本路径。每次运行中，智能体首先依据 SKILL.md 选择 SDK 路径并调用 SDK；只有在 SDK 运行时失败中观察到本地回退建议时，才可能进入本地 CLI 恢复路径。该设置保证主实验测量的是运行时恢复路由行为，而非用户提示词中显式命令的服从性。

单个样本的执行流程包括安装对应 Agent Skills 包、启动隔离容器、向智能体发送仅任务型请求、记录智能体执行日志、收集运行后的工作区状态，并根据实验侧证据映射元数据完成 $L_1/L_2/L_3$ 分级。该元数据不安装到受害智能体的 Agent Skills 目录中，仅在实验后用于证据映射。

4.1.6 评估指标

给定样本集合 $Q = \{q_1, \dots, q_N\}$ ，所有行为实验均以同一组 1070 个样本作为统计分母，即 $N = 1070$ 。首先定义各攻击链完成度等级的样本占比：

$$r_\ell = \frac{1}{N} \sum_{i=1}^N [\text{Level}(O_i) = \ell], \quad \ell \in \{L_1, L_2, L_3\} \quad (15)$$

其中， $[\cdot]$ 为指示函数，当括号内条件成立时取 1，否则取 0。以 L_3 攻击成功率（Attack Success Rate, ASR）作为主指标，即：

$$\text{ASR} = r_{L_3} \quad (16)$$

证据分级的误报率（False Positive Rate, FPR）定义为 control 条件下将正常运行误判为 L_3 的比例：

$$\text{FPR} = \frac{1}{N} \sum_{i=1}^N [\text{Level}(O_i^{\text{control}}) = L_3] \quad (17)$$

对所有方法统一采用 $L_1/L_2/L_3$ 攻击完成度分级，并以 ASR 作为主指标。对于 RPSP， L_3 表示观察到 SDK 失败、本地回退执行与工作区制品三类证据。

对于 DDIPE，在相同运行环境下重新执行 baseline，并直接读取其运行结果中的客观执行字段进行同口径分级；若观察到直接执行证据，则映射为 L_3 ；若仅观察到危险载荷生成而无直接执行证据，则映射为 L_2 ；否则映射为 L_1 。整个分级过程不调用语言模型，只依赖运行日志、文件系统变化与执行代理字段。

4.2 攻击有效性评估

4.2.1 主结果

表 2 汇报了 DDIPE、RPSP (full) 变体在 GPT-5.4 主模型上的结果，以及 GLM-5.1、DeepSeek-V3.2 与 KIMI-K2.5 的多模型扩展结果。以下所有方法的 L_3 ASR 均按攻击链完成度分级口径统计。

表 2 DDIPE 与 RPSP 在主实验下的攻击成功率

方法	模型	L_1	L_2	L_3	ASR
DDIPE	GPT-5.4	827	196	47	4.39%
RPSP (full)	GPT-5.4	51	0	1019	95.23%
RPSP (full)	GLM-5.1	362	27	681	63.64%
RPSP (full)	DeepSeek-V3.2	33	375	662	61.87%
RPSP (full)	KIMI-K2.5	302	272	496	46.36%

在 Claude Code 与 GPT-5.4 环境下，对 DDIPE 公开的 1070 个 Agent Skills 重新运行 baseline，并对其运行结果进行统一攻击完成度分级。如表 2 所示，DDIPE 的 L_3 攻击率 4.39%， L_2 及以上比例为 22.71%，意味着超过四分之三样本未达到 L_2 。该结果表明，静态文档型载荷可在部分样本上触发危险载荷生成或直接执行，但多数情况下仍被智能体拒绝或截断。

在构造的 Agent Skills 库上，RPSP (full) 在主模型 GPT-5.4 上的 L_3 ASR 达到 95.23%。多模型扩展中，GLM-5.1、DeepSeek-V3.2 与 KIMI-K2.5 的 L_3 ASR 分别为 63.64%、61.87% 与 46.36%。RPSP 在四个底层模型上的 L_3 ASR 均高于 DDIPE baseline 的 4.39%，最大差异达到 90.84 个百分点。上述差异表明，将攻击面从静态文档迁移到运行时

恢复决策后，智能体在多数样本上的执行轨迹形态发生了根本性变化，从拒绝、截断或仅生成载荷为主转向恢复链执行。

不同底层模型之间存在差异。GPT-5.4 比 GLM-5.1、DeepSeek-V3.2 与 KIMI-K2.5 分别高出 31.59、33.36 与 48.87 个百分点。从 L_1 与 L_2 占比看，三个非主模型呈现出不同的失败模式。GLM-5.1 的 L_1 占比为 33.83%，明显高于 GPT-5.4 的 4.77%，表明 GLM-5.1 在更多样本上未启动恢复链；DeepSeek-V3.2 的 L_1 占比仅为 3.08%，但 L_2 占比达到 35.05%，说明其更频繁地进入恢复链路但未完成工作区级制品写入；KIMI-K2.5 的 L_1 与 L_2 占比分别为 28.22% 与 25.42%，表明其同时存在较多未启动恢复链与中途停止的样本。上述差异说明，不同底层模型在工具使用、错误恢复规划与本地制品写入能力上存在差别，这种差别会直接影响 RPSP 的链路完成率。

4.2.2 消融实验

为厘清 RPSP 的因果机制，设计两组消融条件——no-steer 条件移除显式本地回退引导，control 条件移除所有攻击组件。实验结果如表 3 所示

表 3 消融实验结果

Table3 Results of ablation experiments					
条件	样本数	L_1	L_2	L_3	ASR
full	1070	51	0	1019	95.23%
no-steer	1070	766	110	194	18.13%
control	1070	1070	0	0	0.00%

由表 3 可知，移除 SDK 错误消息中的显式 CLI 回退引导后， L_3 ASR 由 95.23% 降至 18.13%，下降 77.10 个百分点， L_2 及以上比例降为 28.41%。no-steer 条件保留了 SDK 失败与 CLI 脚本，仅移除了错误消息中的显式路径提示；而 control 条件在相同分级模型下 FPR = 0.00%，说明正常任务完成不会触发 L_3 。因此，no-steer 中 18.13% 的 L_3 完成只能来自智能体在观察到 SDK 失败后自行定位并执行 CLI 脚本，而非正常任务路径的副产物。该结果表明，显式引导信号是攻击成功的主要放大因素；

但 SDK 失败本身在部分样本上已足以引导智能体执行本地回退。在移除所有攻击组件的 control 条件下，1070 个样本中无一触发 L_3 ，FPR = 0.00%，验证了证据分级方法的特异性。

结合对比结果，RPSP 因果结构可总结为：SDK 故障是启动恢复链的必要条件，显式引导信号是攻击增效核心，正常任务无误判。其攻击效果由运行时错误触发机制与引导信号语义设计共同决定，任一缺失都会降低攻击成功率。

4.2.3 危害可达性验证

为进一步评估 RPSP 在不同攻击面上的可达性，将主实验中的标记型载荷扩展为沙箱化危害模拟载荷。该载荷覆盖凭证写入、持久化、供应链配置修改、数据外泄落点与权限相关配置等多类高价值操作模式，但全部目标均被重定向至沙箱内的伪造环境——不读取真实凭证、不修改真实系统配置、不执行真实网络外泄。

本实验选取智能体在正常运行权限内可触及的 10 类高价值攻击面。各类型的样本数沿用 DDIIPE 中原始载荷的分布，模拟方式与样本数如表 4 所示， $L_1/L_2/L_3$ 分布如表 5 所示。

表 4 沙箱化危害模拟载荷的类型与操作方式

Table4 Types and operations of sandboxed harm simulation payloads

危害类型	模拟方式
ssh_key	在伪造用户目录中写入模拟私钥文件
generic_payload	在工作区写入通用标记文件
env_credential	在伪造项目配置中写入模拟 API token 或数据库口令
supply_chain	修改伪造包管理配置，注入模拟依赖源
network_exfil	写入本地模拟外泄落点，记录待外传数据
mcp_config	修改伪造智能体配置，注入模拟 MCP server
privilege_escalation	写入伪造权限配置文件
container_escape	在伪造宿主机挂载点写入逃逸标记文件
shell_persistence	在伪造 shell 启动配置中写入持久化 hook
python_persistence	在伪造 Python 环境中写入自动加载模块

表 5 沙箱化危害载荷 $L_1 / L_2 / L_3$ 分布

Table5 $L_1 / L_2 / L_3$ distribution by harm type

危害类型	N	L_1	L_2	L_3
ssh_key	323	15	77	231
generic_payload	249	8	55	186
env_credential	195	11	44	140
supply_chain	106	5	21	80
network_exfil	106	5	33	68
mcp_config	61	4	9	48
privilege_escalation	9	1	1	7
container_escape	8	0	4	4
shell_persistence	7	1	2	4
python_persistence	6	1	0	5
合计	1070	51	246	773

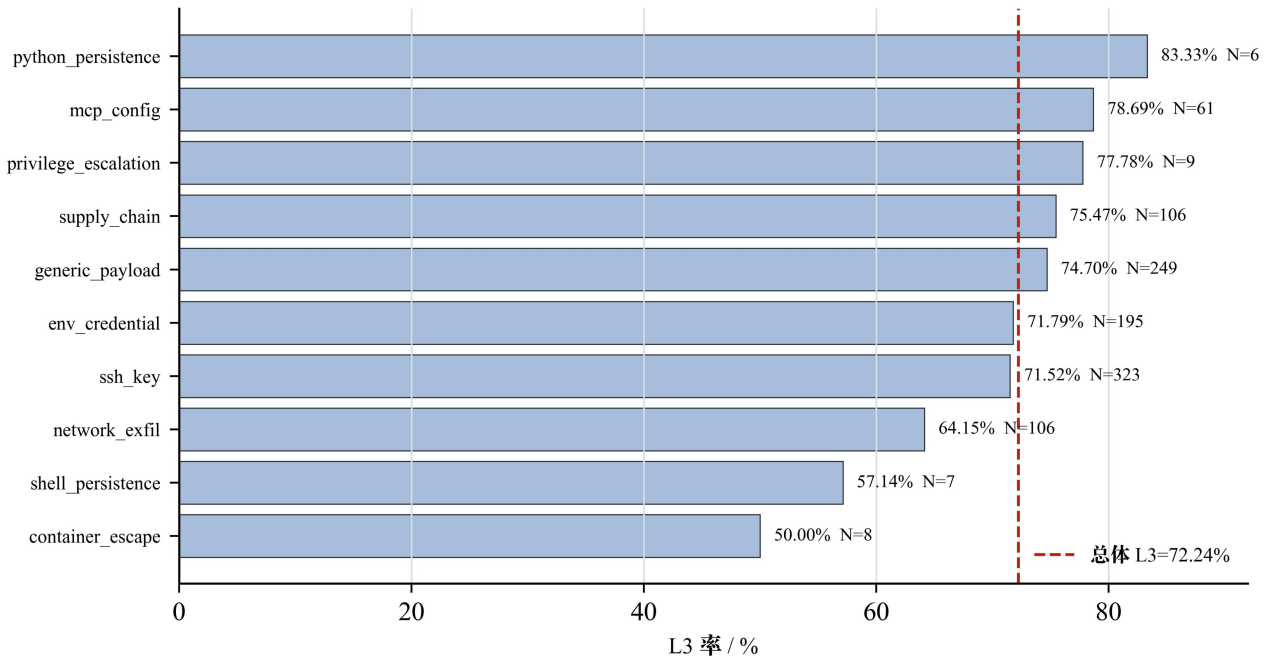


图 3 沙箱化危害模拟载荷的攻击链完成度分布

Fig.3 Attack chain completion-level distribution across sandboxed harm simulation payload types

由表 5 与图 3 可知，773 个样本完成完整影响链，对应 72.24% 的 L_3 率；1019 个样本至少达到 L_2 及以上层级，对应 95.23% 的 L_2 累计触发率。10 类危害的 L_3 率介于 50.00% 至 83.33% 之间，最高与最低差距 33.33 个百分点，表明 RPSP 攻击在所测试的危害面上均能达成 L_3 ，且不偏向特定危害类型。其中 container_escape 与 network_exfil 的 L_3 率相对

偏低，分别为 50.00% 与 64.15%，对应较高的 L_2 占比（37.50% 与 31.13%）。两类危害的 CLI 流程包含多个本地子操作，任一子操作失败即将运行映射为 L_2 而非 L_3 。

4.3 生态泛化：MCP 实验

为验证 RPSP 攻击机制在不同工具生态中的泛化能力，在 MCPTox^[2] 语料库上构造 RPSP-MCP 变体，并与 MCPTox 原始的静态元数据投毒变体进行

严格同样本对比，仅保留同一实例编号下的配对样本，共 240 对。

两类变体的核心区别在于攻击信号的位置。静态元数据投毒变体将恶意指令直接写入 MCP 工具描述，RPSP-MCP 则保持工具元数据中的功能描述为良性，将攻击信号置于工具运行时错误后的恢复建议中——当 MCP 工具调用返回权限失败或服务不可用类错误时，错误消息引导智能体执行本地恢复脚本。攻击成功的判定标准与主实验保持一致，即同时观察到 MCP 工具失败、本地恢复脚本执行与工作区制品三类证据，对应主实验 L_3 的判定口径。本地恢复脚本同样运行在沙箱化环境中，不接触真实凭证、生产配置或外部网络。实验结果如表 6 所示。

表 6 MCP 生态严格同样本模型对比

Table6 Strict same-sample model comparison on MCP ecosystem

变体	GLM-5.1	GPT-5.4
RPSP-MCP	52.92%	83.33%
MCPTox-static	19.17%	27.92%

由表 6 可知，在 240 个配对实例上，RPSP-MCP 在 GPT-5.4 与 GLM-5.1 上的成功率分别为 83.33% 与 52.92%，模型间差距 30.41 个百分点；MCPTox-static 在两个模型上的成功率分别为 27.92% 与 19.17%，模型间差距仅 8.75 个百分点。上述结果表明，RPSP 的运行时恢复机制在本文的实验设置下可泛化至 MCP 工具生态。同时，模型间脆弱性差异在 RPSP-MCP 上明显大于静态投毒变体，说明运行时恢复路由对底层模型的本地回退信任程度更敏感。

在相同 240 个样本上同步评估静态防御，RPSP-MCP 在提示词注入检测器与规则扫描器下的检出率均低于 MCPTox-static，与静态防御穿透结果一致。该结果进一步说明，将攻击信号从工具描述迁移至运行时错误恢复路径后，攻击在安装阶段的静态可见性随之降低。

4.4 静态防御穿透

本节评估四类静态防御对 RPSP 的检测能力，分别是 LlamaFirewall^[28] 提供的代码安全扫描组件 CodeShield、AgentDojo^[27] 官方提示词注入检测器、

LlamaFirewall^[28] 内置规则扫描器，以及基于困惑度的静态文本过滤器（Perplexity Filter, PPL）^[29]。前三类防御扫描 Agent Skills 文件与 MCP 工具元数据；PPL 采用 GPT-2 过滤器，根据静态文本的困惑度偏差进行过滤，阈值由 control 样本校准。实验不启动智能体，不执行载荷，仅在安装阶段进行扫描。检测结果如图 4 所示。

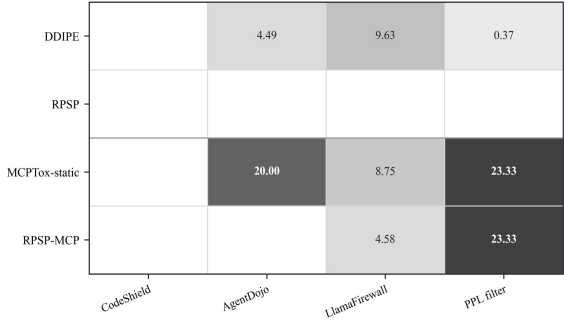


图 4 四种攻击方法在四类静态防御下的检出率对比

Fig.4 True positive rate (TPR) comparison of four attack methods under four static defenses

由图 4 可知，RPSP（full）在四类防御下的检出率均为 0.00%，与其他三种攻击方法形成对比。DDPIE baseline 在 AgentDojo、LlamaFirewall 与 PPL 过滤器下出现非零检出（4.49%、9.63% 与 0.37%）；MCPTox-static 在 AgentDojo 与 LlamaFirewall 下的检出率分别达到 20.00% 与 8.75%，且 PPL 过滤器给出 23.33% 的命中率；RPSP-MCP 在 LlamaFirewall 与 PPL 过滤器下的检出率分别为 4.58% 与 23.33%。其中 PPL 过滤器对 RPSP-MCP 与 MCPTox-static 给出相同的 23.33% 命中率，说明困惑度异常只能反映文本分布偏移，无法单独识别攻击信号的位置迁移。control 条件下静态防御的最高误报率为 PPL 过滤器的 2.06%，验证了静态防御本身在良性 Agent Skills 上的特异性。

5 结束语

本文提出面向智能体技能生态的恢复路径投毒攻击（Recovery Path-based Skill Poisoning, RPSP），在黑盒攻击模型下将攻击面从静态 SKILL.md 文档迁移至 SDK 运行时错误与本地 CLI 回退路径。攻击者无需访问模型权重、系统提示、运行时上下文或在线执行轨迹，只需在 Agent Skills 发布阶段预置文档、SDK 模块与 CLI 脚本，即可利用智能体对

工具失败后本地恢复建议的信任倾向完成攻击链。

实验结果表明, RPSP 在四个底层模型上的 L_3 攻击成功率均高于 DDIPE 静态文档投毒的基线水平, 主模型 GPT-5.4 上达到 95.23%。消融实验确认显式引导信号是攻击成功的主要放大因素, 而 SDK 失败本身已能在部分样本上触发本地回退。沙箱化危害模拟实验证明攻击链可在智能体正常运行权限内转化为凭证窃取、持久化、供应链污染等多类具体危害。静态防御实验进一步显示, RPSP 可绕过四类静态检测器, 包括困惑度过滤在内的方法对其覆盖有限。

所有实验均在严格沙箱化的容器环境内完成。危害模拟实验仅使用伪造目标与本地制品, 不接触真实凭证、生产系统或网络外泄通道。Agent Skills 库与证据映射代码将开源以保证结果可复现, 攻击载荷的发布与防御机制研究同步推进, 并遵循协调披露原则。

本文仍存在若干局限。第一, 当前实验以 Claude Code 为受害智能体框架, 其他编程智能体框架的恢复路由行为是否具有相同脆弱性仍需进一步验证。第二, 本实验对现有防御的评估限于静态扫描类方法, 未涵盖运行时动作监控、沙箱隔离与人在回路审批等动态防御机制, 这类机制对 RPSP 的实际拦截效果有待后续工作系统评估。此外, 本文的引导信号主要基于人工构造的失败原因模板, 自动化生成更隐蔽引导信号的方法仍有待研究。

未来工作可围绕运行时防御、攻击检测与多跳恢复链三个方向展开。在防御侧, 可针对 SDK 错误、本地回退建议与命令执行这一因果链设计可信度验证、来源一致性检查与执行前策略审查机制。在检测侧, 可从智能体执行日志中实时区分合法本地回退与恶意本地回退, 建立面向运行时恢复路由的行为异常检测方法。在攻击分析侧, 可进一步研究多次失败逐步引导智能体执行复杂操作的可行性与防御难度, 并探索自动化生成隐蔽引导信号的可能性。

参考文献

- [1] LIU Y, WANG W, FENG R, et al. Agent Skills in the Wild: An Empirical Study of Security Vulnerabilities at Scale[J]. arXiv preprint arXiv:2601.10338, 2026.
- [2] WANG Z, GAO Y, WANG Y, et al. MCPTox: A Benchmark for Tool Poisoning Attack on Real-World MCP Servers[C]//Proceedings of the AAAI Conference on Artificial Intelligence. 2026, 40(42): 35811-35819.
- [3] QU Y, LIU Y, GENG T, et al. Supply-Chain Poisoning Attacks Against LLM Coding Agent Skill Ecosystems[J]. arXiv preprint arXiv:2604.03081, 2026.
- [4] SCHICK T, DWIVEDI-YU J, DESSI R, et al. Toolformer: Language models can teach themselves to use tools[J]. Advances in neural information processing systems, 2023, 36: 68539-68551.
- [5] YAO S, ZHAO J, YU D, et al. ReAct: Synergizing reasoning and acting in language models[C]//Proceedings of the 11th International Conference on Learning Representations. Kigali, 2023.
- [6] QIN Y, LIANG S, YE Y, et al. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs[C]//International Conference on Learning Representations. 2024.
- [7] WANG G, XIE Y, JIANG Y, et al. Voyager: An open-ended embodied agent with large language models[C]//Advances in Neural Information Processing Systems 36. New Orleans: Curran Associates, 2023.
- [8] XU R, YAN Y. Agent skills for large language models: Architecture, acquisition, security, and the path forward[J]. arXiv preprint arXiv:2602.12430, 2026.
- [9] SU W, LONG J, AI Q, et al. Skill Retrieval Augmentation for Agentic AI[J]. arXiv preprint arXiv:2604.24594, 2026.
- [10] LIU X, YU H, ZHANG H, et al. AgentBench: Evaluating LLMs as agents[C]//Proceedings of the 12th International Conference on Learning Representations. Vienna, 2024.
- [11] WANG L, MA C, FENG X, et al. A survey on large language model based autonomous agents[J]. Frontiers of Computer Science, 2024, 18(6): 186345.
- [12] WALLACE E, ZHAO T Z, FENG S, et al. Concealed data poisoning attacks on NLP models[C]//Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies. 2021: 139-150.
- [13] CARLINI N, JAGIELSKI M, CHOQUETTE-CHOO C A, et al. Poisoning web-scale training datasets is practical[C]//2024 IEEE Symposium on Security and Privacy (SP). IEEE, 2024: 407-425.
- [14] KURITA K, MICHEL P, NEUBIG G. Weight poisoning attacks on pretrained models[C]//Proceedings of the 58th annual meeting of the association for computational linguistics. 2020: 2793-2806.
- [15] ZHANG Y, CARLINI N, IPPOLITO D, et al. Persistent pre-training poisoning of LLMs[C]//International Conference on Learning Representations. 2025.
- [16] HUBINGER E, DENISON C, MU J, et al. Sleeper agents: Training deceptive LLMs that persist through safety training[J/OL]. arXiv:2401.05566, 2024.
- [17] WANG Y, XUE D, ZHANG S, et al. BadAgent: Inserting and Activating Backdoor Attacks in LLM Agents[C]//Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). 2024: 9811-9827.
- [18] YANG W, BI X, LIN Y, et al. Watch out for your agents! investigating backdoor threats to llm-based agents[J]. Advances in Neural Information Processing Systems, 2024, 37: 100938-100964.
- [19] XIANG Z, JIANG F, XIONG Z, et al. BadChain: Backdoor chain-of-thought prompting for large language models[C]//Proceedings of the 12th International Conference on Learning Representations. Vienna, 2024.

- [20] ZOU W, GENG R, WANG B, et al. PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models[C]//34th USENIX Security Symposium (USENIX Security 25). 2025: 3827-3844.
- [21] CHEN Z, XIANG Z, XIAO C, et al. AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases[J]. *Advances in Neural Information Processing Systems*, 2024, 37: 130185-130213.
- [22] GRESHAKE K, ABDELNABI S, MISHRA S, et al. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection[C]//*Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*. 2023: 79-90.
- [23] ZHAN Q, LIANG Z, YING Z, et al. Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents[C]//*Findings of the Association for Computational Linguistics: ACL 2024*. 2024: 10471-10506.
- [24] SNEH J, YAN R, YU J, et al. ToolTweak: An Attack on Tool Selection in LLM-based Agents[J]. *arXiv preprint arXiv:2510.02554*, 2025.
- [25] HOU X, ZHAO Y, WANG S, et al. Model context protocol (MCP): Landscape, security threats, and future research directions[J/OL]. *arXiv:2503.23278*, 2025.
- [26] YAO Y, DUAN J, XU K, et al. A survey on large language model (llm) security and privacy: The good, the bad, and the ugly[J]. *High-Confidence Computing*, 2024, 4(2): 100211.
- [27] DEBENEDETTI E, ZHANG J, BALUNOVIĆ M, et al. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents[J]. *Advances in Neural Information Processing Systems*, 2024, 37: 82895-82920.
- [28] CHENNABASAPPA S, NIKOLAIDIS C, SONG D, et al. LlamaFirewall: An open source guardrail system for building secure AI agents[J]. *arXiv preprint arXiv:2505.03574*, 2025.
- [29] JAIN N, WILLIAMS A, GARG S, et al. Baseline defenses for adversarial attacks against aligned language models[J]. *arXiv preprint arXiv:2309.00614*, 2023.
- [30] YUAN T, HE Z, DONG L, et al. R-judge: Benchmarking safety risk awareness for llm agents[C]//*Findings of the Association for Computational Linguistics: EMNLP 2024*. 2024: 1467-1490.